

Phase 4 Documentation

Knowledge Retrieval & Retrieval-Augmented Generation (RAG)

Intelligent Loan Origination Agent – Retail Banking

This document presents the complete implementation details of Phase 4 of the Intelligent Loan Origination Agent project. The goal of this phase was to enhance the LLM-powered banking assistant using Retrieval-Augmented Generation (RAG), enabling the system to retrieve banking policies and provide grounded, explainable, and accurate responses.

1. Objectives of Phase 4

The primary objectives of this phase were:

- Implement semantic search using embeddings
- Create a vector database using FAISS
- Enable retrieval of banking policies and knowledge documents
- Improve factual accuracy of responses
- Reduce hallucination risks in LLM outputs
- Handle missing information safely

2. RAG Architecture

The architecture of the upgraded system is shown below:

Workflow:

User Query → Embedding Model → FAISS Semantic Search → Retrieved Documents → Prompt Construction → LLM → Final Response

The retrieval layer acts as an external memory system that provides the LLM with relevant banking policies before response generation.

3. Knowledge Base Preparation

A small banking knowledge base was created to simulate organizational loan policies. The documents were stored as text entries and later converted into embeddings for semantic search. **Sample Banking Policies:** Minimum credit score requirements for personal loans Salary document requirements for home loans Manual review requirements for high-value loans Freelancer-specific document requirements Debt-to-income ratio constraints

4. Text Chunking Strategy

Each banking policy was stored as a separate chunk to simplify retrieval and improve semantic matching accuracy. Smaller chunks reduce irrelevant retrieval noise and improve grounding quality.

Future improvements may include: Overlapping chunks Metadata-based filtering Hierarchical retrieval

5. Embedding Model

The project used the **Sentence Transformers all-MiniLM-L6-v2** embedding model. **Reason for Selection:** Lightweight and efficient Good semantic similarity performance Suitable for real-time retrieval applications Works effectively for short banking policy documents

6. Vector Store Implementation (FAISS)

FAISS (Facebook AI Similarity Search) was used as the vector database. **Why FAISS?** Fast semantic similarity search Efficient handling of vector embeddings Simple integration with Python workflows Scalable for large document collections Each document embedding was stored in the FAISS index, allowing semantic retrieval based on vector similarity.

7. Retrieval Quality Testing

The semantic retrieval system was tested using various banking-related user queries. **Sample Test Queries:** “What documents are needed for freelancers?” “What is the minimum credit score required?” “Do high-value loans require manual verification?” The system successfully retrieved the most relevant policy documents for most queries.

8. Comparison: With vs Without Retrieval

Metric	Without Retrieval	With Retrieval
Policy Accuracy	Medium	High
Hallucination Risk	High	Reduced
Explainability	Generic	Grounded
Trustworthiness	Medium	High
Maintainability	Difficult	Easy

9. Failure Modes Identified

The following risks and limitations were observed: Poor retrieval for vague or ambiguous queries
Missing documents in the knowledge base
Irrelevant retrieval results affecting response quality
Chunking issues causing partial information retrieval
Dependency on embedding quality

10. Conclusion

Phase 4 successfully introduced Retrieval-Augmented Generation into the Intelligent Loan Origination Agent. The system can now retrieve banking policies using semantic search and generate grounded responses based on retrieved knowledge. This significantly improves explainability, factual reliability, and compliance compared to the previous LLM-only implementation. The RAG architecture also creates a scalable foundation for future enhancements such as advanced planning, memory systems, and tool integration.